Software Development for Robotics (SDfR) - Assignment V

S.T.C. Fokkema (s2710714) & J.B.E. Nwandu (s3537544) – PPD: Group 1

March 24, 2026

RELbot Block Diagram

The CPS architecture of the RELbot (from Gijs van Oort, et al.) is shown in Figure 1. Note that not all components are ROS2 or even software, but are expressed in the style of a ROS2 system for clarity.

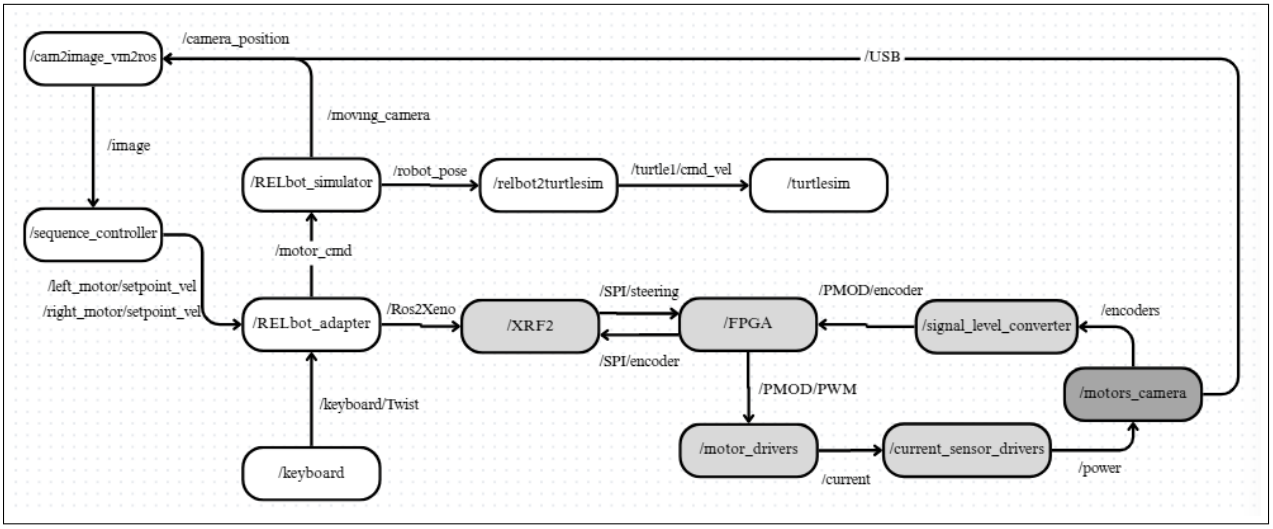

Figure 1: Representation of the CPS architecture in ROS2 format. The white nodes are ROS2 nodes to be programmed in this course. The light grey nodes are other electronic/embedded systems outside the scope of this course and the dark grey node is the physical system which does not include software whatsoever.

All nodes are explained below:

Table 1: Description of the nodes in the CPS architecture

Node	**Hardware**	**Function**
/FPGA	Icoboard (Verilog)	runs the PWM-pulse generation and encoder-pulse counting
/XRF2	Raspberry Pi 4B + Xenomai 4real-time OS extension	tranlsates the objective via loop-control and applies necessary safety margins
/motor_driver	Pololu 2999s	amplify th Pololu 2999se PWM into an electrical power signal (one independent signal per motor)
/current_sensor_driver	Pololu 2452s	registering the current passing throught the motor (one independent signal per moto)
/signal_level_converter	SPI-I2C-UART	registering an encoder count (independently for each motor)
/motors_camera		movement of the robot and allowing measurement of the encoders and the camera to change
/RELbot_adapter	Raspberry Pi 4B (ROS2)	connects the ROS2 nodes to either the simulator or to the real RELbot (depends on the parameter) and does safety checks
/RELbot_simulator	Raspberry Pi 4B (ROS2)	provides a model of the RELbot and a loop controller (SRT software)
/turtlesim	Raspberry Pi 4B (ROS2)	virtually drawing the virtual position of the robot (turtle) of the simulation
/keyboard	Raspberry Pi 4B (ROS2)	send Q or F keyboard interruptions to the running system
/sequence_controller	Raspberry Pi 4B (ROS2)	determines the objective of the robot from the current state by image processing, pathfinding or other tasks
/relbot2turtlesim	Raspberry Pi 4B (ROS2)	implements the conversion from relbot_simulator to turtlesim
/cam2image_vm2ros	Raspberry Pi 4B (ROS2)	captures raw video frames from the camera

All topics are explained below:

Table 2: Description of the topics/communications in the CPS architecture

Topic	**Publisher(s)**	**Subscriber(s)**	**Data type**
/SPI/encoder	/FPGA	/XRF2	encoder count (14bit unsigned integer)
/SPI/steering	/XRF2	/FPGA	two steering percentages (2x float64)
/PMOD/PWM	/FPGA	/motor_drivers	pulse width modulation (PWM) from an 11bit signed integer
/PMOD/encoder	/signal_level_converter	/FPGA	PWM_ENA (no pulse) and PWM_ENB (index pulse) with 4 pulses per encoder count. Each motor produces pulses separately.
/encoders	/motors_camera	/signal_level_converter	movement of the RELbot
/current	/motor_drivers	/current_sensor_drivers	DC current [A]
/power	/current_sensor_drivers	/motors_camera	DC power [W]
/USB	/motors_camera	/cam2image_vm2ros	
/moving_camera	/motors_camera	/cam2image_vm2ros	sensor_msgs/msg/Image
/camera_position	/RELbot_simulator	/cam2image_vm2ros	geometry_msgs/msg/PointStamped
/turtle1/cmd_vel	/relbot2turtlesim	/turtlesim	geometry_msgs/msg/twist_stamped
/output/robot_pose	/RELbot_simulator	/relbot2turtlesim	geometry_msgs/msg/pose_stamped
/motor_cmd	/RELbot_adapter	/RELbot_simulator	relbot_msgs/msg/RelbotMotors ()
/Ros2Xeno	/RELbot_adapter	/XRF2	xrf2_msgs/msg/Ros2Xeno ()
/left_motor/setpoint_vel	/sequence_controller	/RELbot_adapter	example_interfaces/msg/Float64
/right_motor/setpoint_vel	/sequence_controller	/RELbot_adapter	example_interfaces/msg/Float64
/image	/cam2image_vm2ros	/sequence_controller	sensor_msgs/msg/Image
/keyboard/Twist	/keyboard	/RELbot_adapter	geometry_msgs/msg/Twist ()

ROS2 Software Design

The tutorials "Using `turtlesim`, `ros2` and `rqt`", "Understanding nodes", "Understanding topics", "Launching nodes", "Creating a workspace", "Understanding parameters" and "Using parameters in a class (`C++`)" were followed prior to the completion of the mentioned code.

First, a subscriber to the `robot_pose` topic was created, that extracts the current angle and x- & y-positions. With these values, the robot could be more precisely controlled to make 90 degree turns. In the end, however, it seems that it was never truly accurate and always overshot a bit. This is likely due to the somewhat low frequency the messages are transferred at, which means the robot could be at a further angle before its current pose is even transmitted to the subscribers. Figure 2 shows the path traced after each sequence is completed.

Straight Line

The straight line is the most straightforward of all sequences. The left wheel and the right wheel go at the same speed, however the left wheel gets a negative value because the two wheels are identical, and thus they should turn in opposite directions of each other to move forwards. This sequence ends after 5 seconds.

The velocity value given to the wheels at this step is 10 units per second, so over 5 seconds this would be 50 units travelled.

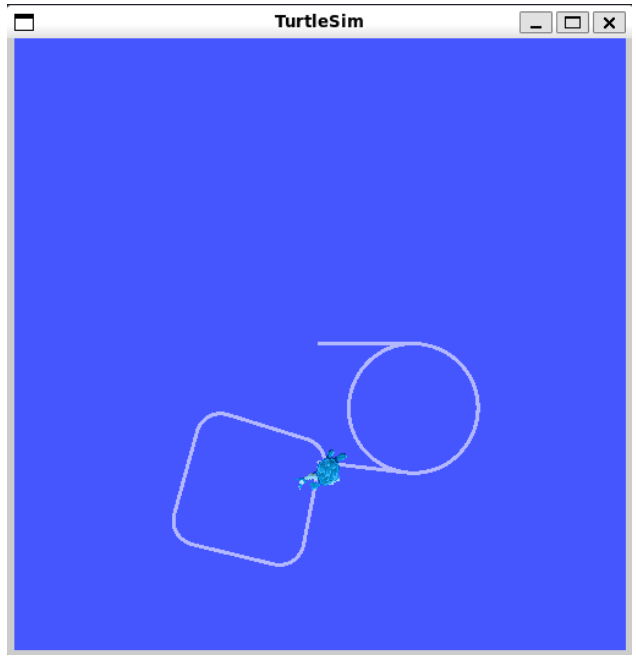

Figure 2: Output of the sequence controller.

Circle

The robot moves in a circle, if the left and right wheels turn at unequal speeds. In this case, the left wheel turns faster than the right one, and thus it will turn to the right. The goal was to have the robot make a full circle first, and then keep going for another half circle so that it ends up facing the left. To achieve this, the angle at the start of this sequence is recorded, and its current angle is constantly compared to it. Once the robot has turned a total of 3π radians, which is exactly a circle and a half, the circle sequence is completed.

Here, the left wheel is given a velocity of 10, and the right wheel a velocity of 8.5 units per second. With that, the radius of the circle R can be computed with $R = \frac{L}{2}\frac{10+8.5}{10-8.5} = \frac{L}{2}\frac{18.5}{1.5} \approx 6.16L$, where L would be the distance between the wheels. This value is not actually known, so taking $L = 5$ we get a radius of 30.8 units, which seems correctly proportionate to the straight line before it.

Left Turn

For the left turn, it first drives forwards for 7 seconds. After that, the wheels must turn in "opposite" directions, which in this case means giving them both positive velocity values. Doing so will make it turn in place. By comparing the robot's current angle to the one that was recorded at the start of this sequence, the cosine of the difference between the two is equal to 0 when this difference is 90°. This measure is used to determine when it should stop turning, and start driving forwards instead.

For the first straight line before the left turn, it travels for 3 seconds at a velocity of 10 units per second. This comes out to be 30 units long. Then, after turning in place (radius of 0), the time it travels can differ from run to run, as sometimes it may take slightly longer to start driving again due to the message frequency.

Square-like Path

For the square-like path, the robot's current position is recorded each time it completes a corner. After this, it drives forwards until the total distance travelled from this corner reaches a certain amount. Once that happens, it starts making the rounded corner. This is done in the same way as the circle is drawn, except

this time the ratio between the speed of the inner and outer wheel is larger, making the radius of the circle smaller. In this case, the robot will stop after it has turned a total of $\pi/2$ radians. Once it has finished this corner, it records its current position again and does the same process over again. Each time it completes a corner, a counter is incremented by 1. Once this counter reaches 4, it means every corner is finished and the robot finally exits the sequence and stops.

For the straight sections, it is hard coded that the robot starts turning after 1.5 meters. Clearly, this is a different unit than the generic "units" that were used previously. The conversion from the angular velocity units to meters can only be made if the radius of the wheels is known. As for the radius of the corners, the same formula for the circle can be applied, with an inner wheel velocity of 2 and an outer wheel velocity of 3 units per second: $R = \frac{L}{2}\frac{3+2}{3-2} = \frac{L}{2}\frac{5}{1} = 2.5L$. Again, taking $L = 5$, the radius is 12.5 units.

AI Disclosure

During the preparation of this work the authors used `MS Copilot` in order to improve code and syntax. After using this tool/service, the authors reviewed and edited the content as needed and take full responsibility for the content of the delivered work.